

Harshith Kantamneni

Full-time roles in GPU performance, LLM inference, and AI infrastructure

kantamneniharshith@gmail.com | +1 (414) 916-5799 | linkedin.com/in/hk4231
github.com/harshithkantamneni | harshithkantamneni.github.io | Milwaukee, WI

SUMMARY

AI systems engineer, silicon to agents. Optimizes CUDA/Triton kernels and profiles LLM inference at the metal (**215 TFLOP/s**, 69% of A100 fp16 peak), and designs the autonomous multi-agent, retrieval, and evaluation systems on top.

TECHNICAL SKILLS

GPU, Architecture & Performance: CUDA, Triton (kernel DSL), PyTorch, kernel optimization, kernel fusion, Tensor Cores, mixed precision (fp16), GPU memory hierarchy, roofline/memory-bandwidth analysis, NVIDIA Nsight Systems/Compute, quantization, ROCm/HIP, gem5, McPAT

AI Systems: LLM inference profiling, TensorRT, FlashAttention/SDPA, KV-cache decode, RAG (retrieval-augmented generation), multi-agent orchestration, Model Context Protocol (MCP) server design, Claude Code / Agent SDK, agent evaluation

Retrieval & Memory: Hybrid search, reciprocal-rank fusion (RRF), cross-encoder reranking, sqlite-vec, SQLite FTS5

Languages & Infra: Python, C++, C, Docker, CI/CD, Slurm

EDUCATION

University of Wisconsin-Madison

M.S. Electrical and Computer Engineering

Madison, USA

Sep 2024 - Dec 2025

- Relevant Coursework: High Performance Computing, Advanced Computer Architecture, Machine Learning, Fault-Tolerant Computing

Vellore Institute of Technology

B.Tech in Electronics and Communication Engineering

Amaravati, India

2020 - 2024

PROJECTS & EXPERIENCE

Triton vs cuBLAS LLM Kernel Benchmarking

Triton, CUDA, PyTorch, NVIDIA Nsight

Dec 2025 - Present

github.com/.../triton-vs-cublas-llm-benchmarks

- Fused linear+bias+**GELU** into one **Triton** kernel, cutting p50 latency **up to 1.73x** over the **unfused** cuBLAS-GEMM + separate-launch path on small-batch memory-bound FFN. The win is removing an **HBM** round-trip and two launches, not a faster GEMM (median 0.96x at larger batch).
- Benchmarked **76 LLM-shaped** GEMM kernels across Triton and cuBLAS on an **A100** (p50/p90 latency, throughput, bandwidth, tail jitter) to map where fusion wins. The autotuned hand-tiled Triton GEMM sustains **215 TFLOP/s**, **69% of fp16 tensor-core peak** (cuBLAS reaches 79% on the same shapes).

Edge LLM GPU Profiling on Jetson Orin Nano

CUDA, NVIDIA Nsight Systems, PyTorch, FlashAttention

Feb 2026

github.com/.../edge-llm-gpu-profiling

- Profiled **attention** and **KV-cache decode** on a Jetson Orin Nano (15 W edge GPU) with **Nsight Systems**, tracing the per-token **kernel-launch overhead** that dominates the decode loop.
- Measured **15.4K tok/s** on a single-stream KV-cache decode microbenchmark (synthetic B=1, H=8, D=64 shapes, not full-model decode) and **7.5 TFLOP/s** peak fp16 GEMM, with fused **SDPA** (scaled dot-product attention) **5.6x** over a naive materialized-attention reference.

bert: Long-Context Memory for AI Coding Assistants (MCP)

May 2026 - Present

Python, MCP, sqlite-vec, sentence-transformers

github.com/.../bert

- Engineered **bert**, a local **Model Context Protocol (MCP)** server that gives AI coding assistants (Claude Code, Cursor, Codex) a searchable memory of an entire codebase via **hybrid retrieval** (dense vector + keyword, cross-encoder reranked), pulling only the relevant files into the model's context.
- Root-caused a silent retrieval-fusion bug that had pinned a held-out QA eval near-random (**0.10**) and restored it to **0.85**. On **BEIR** scifact the reranked hybrid scores **0.745 nDCG@10**, beating the published BM25 baseline (+**0.08**) and matching the bge-base reference.
- Built a held-out QA benchmark on a **3M-token** corpus (3x a **1M** window) where full-context is infeasible: truncation scores **0.00** while bert holds **0.75** on a small set at a flat **3.3K** tokens/query, showing retrieval is the only path past the window.

ML-Guided CUDA Kernel Configuration

Jan 2025 - May 2025

UW-Madison course project | Python, PyTorch, CUDA, Slurm

github.com/.../ML-Guided-CUDA-Config

- Trained a PyTorch MLP runtime surrogate (12 GEMM/launch features) that predicts CUDA kernel execution time at **$R^2 = 0.96$** (MAE 0.018 ms), replacing exhaustive autotuning with argmin-over-grid config selection.
- Validated predicted block/grid configs to **within 5% of the exhaustively-measured optimum** (median across workloads), with benchmarking and CUDA event-timing automated on Slurm.

MI300X Memory-System Characterization & gem5 Modeling

Jan 2025 - May 2025

UW-Madison course project | HIP, gem5, McPAT, Python

- Characterized the **AMD MI300X** memory hierarchy with **HIP** microbenchmarks (vL1D 130, L2 230, HBM 630-760-cycle latencies, plus **4.6-4.8 TB/s** write bandwidth) and tuned full-system **gem5** to **within 5% MAPE** of measured vL1D latency on the dominant access stride, down from **89%**.
- Compared in-order and out-of-order CPU models in gem5 across SAXPY/DAXPY/IAXPY for CPI, latency, and power, with **McPAT** stats-to-power extraction.

Autonomous ML Research Lab (Multi-Agent System)

Mar 2026 - Present

Python, C, Apple Metal, Claude API

github.com/.../agi-lab-framework

- Designed and operate an autonomous system of **30 role-specialized agents** running an end-to-end **ML research loop** on a single 18GB laptop: agents plan experiments, write and run the code, and review each other's results, with a human only setting direction.
- Architected and code-reviewed the agents' core deliverable, a **from-scratch C17 training engine** (hand-written autodiff, a **Mixture-of-Experts (MoE)** transformer, 4-bit **quantization-aware training (QAT)**, Accelerate/AMX matmul) with **no PyTorch or TensorFlow**, plus hand-written **Apple Metal** GPU kernels unit-tested as ready-to-enable but currently dormant.
- Caught an agent forge 5 approvals, then designed the integrity layer that stopped it: pre-registration, phase gates, and **cryptographic sign-off verification**, with **zero recurrence across 23 runs**.

HIVE: Autonomous Multi-Agent Software Org

Mar 2026 - Present

Rust, libp2p, Python, Claude Code

- Run **HIVE**, an autonomous software org of **45 role-specialized agents** (Director, Evaluator, Red Team) across **203 plan-build-evaluate cycles** on a single product, and authored the governance layer that coordinates them (signed operator values, a tiered reversibility-based decision protocol, JSON hand-off contracts).
- Directed the agents to build a **62K-line Rust** peer-to-peer reasoning engine gated by **1,300+ tests**: a forward-chaining **Rete** inference engine with truth-maintenance, a **16-module differential-privacy** layer (Rényi-DP, k-anonymity), and a **libp2p** stack (Kademlia DHT, Gossipsub).